System Design Learning Roadmap

Timeline Overview

This roadmap helps you master system design from foundational concepts to advanced systems, with project-based and AI-accelerated strategies.

Phase 1: Foundation Building (Months 1-2)

Month 1: Core Concepts & Architecture Fundamentals

- **Design basics:** Monoliths vs microservices, scalability, availability, reliability
- **Performance:** Latency vs throughput
- **Components:** Load balancers, caching layers, databases (SQL vs NoSQL), message queues, CDNs

Month 2: Essential Design Frameworks

- **Distributed systems:** CAP theorem, replication, sharding, consistency models
- **API basics:** RESTful principles, authentication, authorization
- **System design procedure:** Gathering requirements, estimating capacity, high-level diagrams

Mini Projects:

- URL Shortener (theoretical)
- Basic cache layer
- Familiar app mapping (like Twitter)

Phase 2: Intermediate Concepts (Months 3-4)

Advanced Scalability Patterns

- Sharding, replication (strategies and pitfalls)
- Database read/write optimization
- Rate limiting, throttling
- Event-driven processing, message buses

Distributed System Concepts

- Consensus (CAP, PACELC, Paxos, Raft)
- Asynchronous patterns, retries, circuit breakers
- Monitoring, logging, and observability

Essential Design Patterns

- Service discovery/registry, gateways
- Sagas & event sourcing (transaction strategies)
- CQRS (separate write/read paths)

Company-Specific Studies

- Google: Mega-scale search & storage
- Amazon: Microservices & scaling retail
- Facebook: Social graph & real-time chat
- Uber: Location matchmaking
- Netflix: Stream delivery and resilience

Projects:

- Distributed cache system (with eviction)
- Real-time chat app (WebSocket, queue)
- Rate limiter service

Phase 3: Advanced Design (Months 5-6)

Large Systems and Specializations

- **Search engines:** Crawling, inverted indexes, relevance scoring
- **Recommendation engines:** Collaborative filtering basics
- **Collaborative editing:** Locking, version control, real-time presence
- **Streaming analytics:** Handling TBs of real-time data

Advanced Topics

- Eventual consistency, multi-region deployments
- Disaster recovery and business continuity
- Security, compliance, cost optimization

Build:

- Scalable crawler/search engine
- Stock trading platform (matching engine, risk)
- Social network (relationships, notification)
- Machine learning serving infrastructure

Resources

Premium

- *Grokking the System Design Interview* (Educative.io)
- *System Design Interview* (Alex Xu book/ByteByteGo)
- *Arpit Bhayani System Design Masterclass*
- *DesignGurus Mock Interviews*

Free

- **System Design Primer** (GitHub)
- **Designing Data-Intensive Applications** (Kleppmann)
- **GeeksforGeeks System Design**
- **Awesome System Design** (GitHub)
- **High Scalability Blog**
- YouTube: Academind & System Design playlists

AI for System Design Acceleration

1. Decompose Hard Problems

Prompt AI to break requirements, functional needs, and clarify trade-offs.
Example: “Break down the requirements and challenges for building WhatsApp.”

2. Solution Brainstorming

Ask AI for multiple high-level architectures, with pros/cons and cost/performance impacts.

3. Gap Drilling & Rapid Explanation

Request concise beginner explanations, real analogies, real-world pitfalls, and practice questions on demand.

4. Mock Interviews and Whiteboards

Practice with AI-as-interviewer for system design (timed, with critique and follow-ups).

5. Starter Code and Docs

Generate stubs for APIs, architecture diagrams, documentation, and structured tech trade-off tables.

6. Company-Specific Patterns

Prompt AI: “Explain Instagram feed architecture for 1B users, list pros/cons, and likely interview follow-ups.”

Milestone Projects (with C++ Focus)

1. **URL Shortener Service**
2. **Distributed Cache (Mini Redis)**
3. **Chat Application (real-time)**
4. **Rate Limiter Service**
5. **Search Engine with Web Crawler**
6. **Social Network Platform**
7. **Stock Trading Platform**
8. **Video Streaming Backend**
9. **Machine Learning Model Serving API**
10. **Database Engine Core**

Weekly Routine Example

- Day 1-3: Learn 1-2 concepts, design sketches, AI Q&A
- Day 4-5: Code one project feature, profile & optimize it
- Day 6: Mock interview (AI/peer) and system teardown
- Day 7: Refactor code, document trade-offs, track learnings

Final Advice

- Use interactive courses + open-source project blueprints
- Leverage AI (ChatGPT, Gemini, Claude, Perplexity) for design reviews, code stubs, and critique
- Practice regular mock interviews (use paid mock circuits by FAANG ex-interviewers in late stages)
- Build and document 4-5 complete systems—prefer C++ for core plus Python when APIs/scripts needed

---

Your 1-Year System Design Success Path

1. **Start** with core theory and a tiny project (month 1)
2. **Expand** to multi-user, distributed projects (months 3-6)
3. **Polish** interview skills, finish 3+ advanced projects (months 6-9)
4. **Master** system trade-offs, communicate clearly, and iterate (months 10-12)
5. **Integrate AI deeply** through every stage for feedback and acceleration

---

Good Luck—Design, Build, Scale, and Succeed!